

13 - Funciones de orden superior

En JavaScript, las funciones de orden superior son uno de los conceptos más poderosos y útiles. Estas funciones te permiten escribir código más flexible, modular y reutilizable. Pero ¿qué significa realmente que una función sea de orden superior? Simplemente, una función de orden superior es aquella que:

- Recibe una función como parámetro, o
- Devuelve una función como resultado.

Este concepto puede parecer abstracto al principio, pero es una herramienta increíblemente útil para simplificar la lógica en el código. Vamos a desglosarlo de manera amena y clara, con ejemplos para que entiendas su utilidad práctica.

¿Por qué son útiles las funciones de orden superior?

Las funciones de orden superior son útiles porque:

- **Reutilización de código:** Permiten que funciones genéricas realicen tareas personalizables sin necesidad de escribir la misma lógica varias veces.
- **Modularidad:** Separan el comportamiento general de una función del comportamiento específico, lo que hace el código más limpio y fácil de entender.
- **Flexibilidad:** Pueden adaptarse fácilmente a diferentes contextos y situaciones.
- **Composición:** Permiten combinar pequeñas funciones para crear comportamientos más complejos de forma más clara y mantenible.

En pocas palabras, las funciones de orden superior facilitan la creación de código eficiente, claro y adaptable.

1. Funciones que reciben otras funciones como parámetro

El primer tipo de función de orden superior son aquellas que reciben otras funciones como argumento. Esto permite crear funciones que no necesitan saber exactamente qué hacer en cada situación, ya que el comportamiento específico puede pasarse como parámetro.

Ejemplo: Procesar Números

Supongamos que tienes un array de números y quieres realizar una operación con cada uno de ellos. En lugar de escribir una función diferente para cada operación, puedes escribir una función general que reciba otra función como parámetro.

```
function procesarNumeros(array, funcion) {
  let resultado = [];
  for (let i = 0; i < array.length; i++) {
    resultado.push(funcion(array[i])); // Se aplica la función a cada elemento
  }
  return resultado;
}

function duplicar(num) {
  return num * 2;
}

let numeros = [1, 2, 3, 4, 5];
let numerosDuplicados = procesarNumeros(numeros, duplicar);

console.log(numerosDuplicados); // [2, 4, 6, 8, 10]
```

En este ejemplo, la función `procesarNumeros` no sabe ni le importa qué hace la función que recibe. Solo la aplica a cada número del array. Al pasarle duplicar, obtenemos un array con todos los números duplicados.

Otro ejemplo: Funciones anónimas

Podemos hacer que el comportamiento sea aún más dinámico usando funciones anónimas, es decir, funciones que se definen en el momento.

```
let numerosTriplicados = procesarNumeros(numeros, function(num) {  
  return num * 3;  
});  
  
console.log(numerosTriplicados); // [3, 6, 9, 12, 15]
```

En lugar de definir una función separada como duplicar, pasamos directamente una función anónima que triplica los números. Esto añade mucha flexibilidad, ya que no tenemos que crear una nueva función para cada operación que queramos realizar.

2. Funciones que devuelven funciones

El segundo tipo de función de orden superior son aquellas que devuelven otra función como resultado. Esto es muy útil para crear funciones más específicas o configuradas en función de los parámetros.

Ejemplo: Crear Saludos Personalizados

Imagina que quieres crear diferentes tipos de saludos según la formalidad. En lugar de escribir una función para cada saludo, puedes escribir una función general que devuelva una función específica según el tipo de saludo.

```
function crearSaludo(tipo) {
  return function(nombre) {
    if (tipo === "formal") {
      console.log("Buenos días, " + nombre + ".");
    } else {
      console.log("Hola, " + nombre + "!");
    }
  };
}

let saludoFormal = crearSaludo("formal");
let saludoInformal = crearSaludo("informal");

saludoFormal("Carlos"); // "Buenos días, Carlos."
saludoInformal("Ana"); // "Hola, Ana!"
```

Aquí, crearSaludo devuelve una función personalizada basada en el tipo de saludo que queremos. Esto es útil cuando queremos generar comportamientos diferentes sin tener que escribir muchas funciones individuales. Es como tener una fábrica de funciones que crea las funciones necesarias según lo que necesitamos.

Métodos Avanzados de Arrays en JavaScript: map, filter, find, y más

Trabajar con arrays en JavaScript es muy común, y para hacer más fácil y eficiente esta tarea, JavaScript nos ofrece varios métodos avanzados que nos permiten manipular arrays de una forma poderosa y sencilla. Aquí te presentaré los más comunes y útiles, como map, filter, find, some, every y reduce. ¡Y no te preocupes, lo veremos con ejemplos prácticos utilizando arrays de objetos!

1. map()

El método map() nos permite transformar cada elemento de un array y devolver un nuevo array con los resultados. Piensa en map como una manera de recorrer

un array y aplicar una función a cada elemento, pero sin modificar el array original.

Ejemplo: Array de objetos con map

Imagina que tenemos un array de objetos que representan personas, y queremos crear un array con sus nombres en mayúsculas:

```
let personas = [
  { nombre: "Carlos", edad: 30 },
  { nombre: "Ana", edad: 25 },
  { nombre: "Lucía", edad: 35 }
];

let nombresEnMayusculas = personas.map(function(persona) {
  return persona.nombre.toUpperCase();
});

console.log(nombresEnMayusculas); // ["CARLOS", "ANA", "LUCÍA"]
```

Aquí, map toma cada objeto persona, accede a su propiedad nombre, la transforma en mayúsculas y devuelve un nuevo array con esos valores.

2. filter()

filter() nos permite filtrar un array según una condición que nosotros definimos. Nos devuelve un nuevo array con los elementos que cumplen esa condición. ¡Ideal para buscar elementos específicos!

Ejemplo: Filtrar personas mayores de 30 años

```
let personasMayoresDe30 = personas.filter(function(persona) {
  return persona.edad > 30;
});
```

```
console.log(personasMayoresDe30); // [{ nombre: "Lucía", edad: 35 }]
```

En este ejemplo, filter toma cada persona y evalúa si su propiedad edad es mayor que 30. Si lo es, la incluye en el nuevo array.

3. find()

find() es similar a filter, pero en lugar de devolver un array con todos los elementos que cumplen la condición, devuelve el primer elemento que la cumpla. Si no encuentra ningún elemento, devuelve undefined.

Ejemplo: Buscar a una persona con nombre "Ana"

```
let personaAna = personas.find(function(persona) {  
  return persona.nombre === "Ana";  
});  
  
console.log(personaAna); // { nombre: "Ana", edad: 25 }
```

Aquí, find busca el primer objeto en el array donde el nombre sea "Ana". Si existieran más personas llamadas "Ana", find solo devolvería la primera.

4. some()

some() nos dice si al menos un elemento en el array cumple con una condición. Devuelve true si algún elemento cumple la condición, y false si ninguno lo hace.

Ejemplo: Verificar si hay alguien menor de 25 años

```
let hayMenoresDe25 = personas.some(function(persona) {
```

```
    return persona.edad < 25;
  });

console.log(hayMenoresDe25); // false
```

En este caso, some verifica si alguna persona en el array tiene menos de 25 años. Como ninguna cumple esa condición, devuelve false.

5. every()

every() es lo opuesto a some(). Verifica si todos los elementos del array cumplen con una condición. Si todos cumplen, devuelve true; si al menos uno no la cumple, devuelve false.

Ejemplo: Verificar si todas las personas son mayores de 20 años

```
let todosMayoresDe20 = personas.every(function(persona) {
  return persona.edad > 20;
});

console.log(todosMayoresDe20); // true
```

Aquí, every devuelve true porque todas las personas del array tienen más de 20 años.

6. reduce()

reduce() es un poco más avanzado. Se utiliza para reducir todos los elementos de un array a un solo valor. Se le pasa una función que va acumulando un resultado a medida que recorre el array. Puedes usarlo, por ejemplo, para sumar valores o concatenar strings.

Ejemplo: Sumar las edades de todas las personas

```
let edadTotal = personas.reduce(function(accumulator, persona) {  
  return accumulator + persona.edad;  
}, 0);  
  
console.log(edadTotal); // 90
```

Aquí, reduce toma cada persona, suma su edad al acumulador y devuelve la suma total de las edades.

Ejemplo Completo con Todos los Métodos

Para que lo veas en acción, aquí tienes un ejemplo que combina todos los métodos:

```
let personas = [  
  { nombre: "Carlos", edad: 30 },  
  { nombre: "Ana", edad: 25 },  
  { nombre: "Lucía", edad: 35 },  
  { nombre: "Juan", edad: 20 }  
];  
  
// 1. Obtener Los nombres en mayúsculas  
let nombresEnMayusculas = personas.map(persona =>  
  persona.nombre.toUpperCase());  
  
// 2. Filtrar personas mayores de 30 años  
let mayoresDe30 = personas.filter(persona => persona.edad > 30);  
  
// 3. Encontrar a La persona Llamada "Ana"  
let personaAna = personas.find(persona => persona.nombre ===  
  "Ana");  
  
// 4. Verificar si hay alguien menor de 25 años  
let hayMenoresDe25 = personas.some(persona => persona.edad < 25);
```



```
// 5. Verificar si todas las personas tienen más de 20 años
let todosMayoresDe20 = personas.every(persona => persona.edad > 20);

// 6. Sumar todas las edades
let edadTotal = personas.reduce((acumulador, persona) => acumulador + persona.edad, 0);

console.log(nombresEnMayusculas); // ["CARLOS", "ANA", "LUCÍA", "JUAN"]
console.log(mayoresDe30); // [{ nombre: "Lucía", edad: 35 }]
console.log(personaAna); // { nombre: "Ana", edad: 25 }
console.log(hayMenoresDe25); // true
console.log(todosMayoresDe20); // true
console.log(edadTotal); // 110
```

Los métodos avanzados de arrays como map, filter, find, some, every, y reduce son herramientas fundamentales que nos permiten manejar de manera muy efectiva los datos en JavaScript. Estos métodos hacen que nuestro código sea más limpio, fácil de entender y más poderoso, ya que nos permiten hacer transformaciones y evaluaciones complejas con solo unas pocas líneas de código.

Conclusión:

¿Por qué usar funciones de orden superior?

Las funciones de orden superior son una herramienta crucial en JavaScript porque nos permiten:

- Abstraer lógica repetitiva y enfocarnos en lo que realmente importa.
- Modularizar nuestro código, haciéndolo más limpio, fácil de mantener y reutilizable.
- Adaptar el comportamiento de una función sin cambiar su implementación interna.

- Componer comportamientos complejos de manera clara y legible.

Piensa en las funciones de orden superior como piezas de un rompecabezas que puedes combinar de diferentes formas para resolver problemas más grandes. Te permiten crear soluciones flexibles y eficientes que se adaptan a cualquier situación. ¡Y eso es lo que hace a JavaScript un lenguaje tan poderoso y versátil!